R Niveau I

R Niveau I — Exercices Jour 1

Module 1 : Premiers pas avec R

Exercice 1.1 : La console comme calculatrice [Difficulté : Facile]

Énoncé : Utilisez la console R pour calculer les expressions suivantes :

1. 45 + 32
2. 256 / 8
3. 2 puissance 10
4. Le reste de la division de 17 par 5
5. La division entière de 17 par 5

Indice : L'opérateur puissance en R est `^`. Le modulo se note `%%` et la division entière `%/%`.

Solution

Exercice 1.2 : Créer et manipuler des variables [Difficulté : Facile]

Énoncé :

1. Créez une variable `prix_ht` contenant la valeur 85.50
2. Créez une variable `tva` contenant 0.20
3. Calculez le prix TTC et stockez-le dans `prix_ttc`
4. Affichez le prix TTC

Indice : L'opérateur d'assignation en R est `<-`. Le prix TTC = prix HT × (1 + TVA).

Solution

Exercice 1.3 : Écrire un script complet [Difficulté : Facile]

Énoncé : Écrivez un script R qui :

1. Définit le rayon d'un cercle (`rayon <- 7`)
2. Calcule le périmètre (2 × pi × r)
3. Calcule l'aire (pi × r²)
4. Affiche les deux résultats avec `cat()`

Indice : La constante pi est disponible dans R via `pi`. Utilisez `cat()` pour un affichage formaté avec `\n` pour les retours à la ligne.

Solution

Exercice 1.4 : Utiliser l'aide [Difficulté : Facile]

Énoncé :

1. Consultez l'aide de la fonction `round()`
2. Arrondissez la valeur 3.14159265 à 2 décimales
3. Arrondissez la même valeur à 4 décimales
4. Utilisez `floor()`, `ceiling()` et `trunc()` sur la valeur -3.7 et comparez les résultats

Indice : Tapez `?round` ou `help(round)` pour accéder à l'aide. Le paramètre `digits` contrôle le nombre de décimales.

Solution

Exercice 1.5 : Environnement de travail [Difficulté : Intermédiaire]

Énoncé :

1. Créez les variables `a <- 10`, `b <- 20`, `c <- 30`
2. Listez les objets présents dans l'environnement avec `ls()`
3. Supprimez la variable `b` avec `rm()`
4. Vérifiez que `b` n'existe plus avec `exists()`
5. Affichez le répertoire de travail courant

Indice : `rm()` supprime un objet, `exists()` prend le nom de la variable entre guillemets, et `getwd()` affiche le répertoire courant.

Solution

Module 2 : Structures de données

Exercice 2.1 : Types de données fondamentaux [Difficulté : Facile]

Énoncé :

1. Créez une variable de chaque type : `numeric`, `integer`, `character`, `logical`, `complex`
2. Vérifiez le type de chaque variable avec `class()` et `typeof()`
3. Testez si chaque variable est numérique avec `is.numeric()`

Indice : Pour créer un integer explicite, ajoutez `L` après le nombre (ex: `5L`). Un complexe s'écrit avec `i` (ex: `3+2i`).

Solution

Exercice 2.2 : Création de vecteurs [Difficulté : Facile]

Énoncé : Créez les vecteurs suivants :

1. `v1` : les nombres de 1 à 20
2. `v2` : les nombres pairs de 2 à 30
3. `v3` : une séquence de 0 à 1 par pas de 0.1
4. `v4` : le mot "R" répété 5 fois
5. `v5` : les valeurs c(4, 8, 15, 16, 23, 42)

Affichez la longueur de chaque vecteur.

Indice : Utilisez `1:20` pour des séquences simples, `seq()` pour des pas personnalisés, et `rep()` pour la répétition.

Solution

Exercice 2.3 : Indexation de vecteurs [Difficulté : Intermédiaire]

Énoncé : Soit le vecteur `notes <- c(12, 15, 8, 17, 14, 9, 11, 16, 7, 13)`.

1. Extrayez la 3e note
2. Extrayez les notes de la position 2 à 5
3. Extrayez la première et la dernière note
4. Extrayez toutes les notes sauf la 4e
5. Extrayez les notes supérieures ou égales à 12 (indexation logique)
6. Combien d'étudiants ont eu la moyenne (>= 10) ?

Indice : L'indexation négative exclut des éléments. L'indexation logique utilise une condition dans les crochets. `sum()` sur un vecteur logique compte les `TRUE`.

Solution

Exercice 2.4 : Opérations vectorisées [Difficulté : Intermédiaire]

Énoncé : Soit `temperatures_c <- c(0, 10, 20, 25, 30, 37, 100)` (en degrés Celsius).

1. Convertissez en Fahrenheit : F = C x 9/5 + 32
2. Identifiez les températures Celsius supérieures à 25°C
3. Remplacez toute température Celsius supérieure à 37 par la valeur 37 (plafonnement)
4. Calculez la différence entre chaque température successive avec `diff()`

Indice : Les opérations arithmétiques sur un vecteur s'appliquent élément par élément. Pour le plafonnement, utilisez l'indexation logique pour l'assignation.

Solution

Exercice 2.5 : Matrices [Difficulté : Intermédiaire]

Énoncé :

1. Créez une matrice 3x4 contenant les nombres de 1 à 12, remplie par ligne
2. Affichez les dimensions de la matrice

3. Extrayez l'élément à la ligne 2, colonne 3
4. Extrayez la 2e ligne complète
5. Extrayez la 3e colonne complète
6. Calculez la somme de chaque colonne et de chaque ligne
7. Nommez les lignes ("L1", "L2", "L3") et les colonnes ("C1", "C2", "C3", "C4")

Indice : `matrix()` avec `byrow = TRUE` remplit par ligne. `colSums()` et `rowSums()` calculent les sommes.

Solution

Exercice 2.6 : Listes [Difficulté : Intermédiaire]

Énoncé : Créez une liste `etudiant` contenant :

- `nom` : "Dupont"
- `prenom` : "Marie"
- `age` : 22
- `notes` : c(14, 16, 12, 18, 15)
- `redoublant` : FALSE

Puis :

1. Affichez la structure de la liste avec `str()`
2. Accédez au prénom avec `$` et avec `[[ ]]`
3. Calculez la moyenne des notes
4. Ajoutez un élément `moyenne` à la liste
5. Affichez le nombre d'éléments de la liste

Indice : On accède aux éléments d'une liste avec `$nom` ou `[["nom"]]`. `str()` affiche la structure complète d'un objet.

Solution

Exercice 2.7 : Data frames [Difficulté : Intermédiaire]

Énoncé : Créez un data frame `classe` avec les colonnes suivantes :

- `nom` : c("Alice", "Bob", "Clara", "David", "Emma")
- `maths` : c(15, 12, 17, 9, 14)
- `francais` : c(14, 16, 13, 11, 18)
- `sport` : c(12, 18, 10, 15, 16)

Puis :

1. Affichez la structure du data frame
2. Affichez un résumé statistique
3. Accédez à la colonne `maths`
4. Affichez les données d'Emma (5e ligne)
5. Ajoutez une colonne `moyenne` (moyenne des 3 matières pour chaque élève)
6. Triez le data frame par moyenne décroissante

Indice : `str()` et `summary()` sont vos alliés. La moyenne par ligne se calcule avec `rowMeans()` sur les colonnes numériques. `order()` avec `decreasing = TRUE` pour trier.

Solution

Exercice 2.8 : Facteurs [Difficulté : Intermédiaire]

Énoncé :

1. Créez un vecteur `tailles` contenant : "M", "S", "L", "XL", "M", "S", "S", "L", "M", "XL"
2. Convertissez-le en facteur
3. Affichez les niveaux (levels) du facteur
4. Créez un facteur ordonné avec les niveaux dans l'ordre : S < M < L < XL
5. Affichez le tableau de fréquences avec `table()`
6. Vérifiez que S < M donne TRUE avec le facteur ordonné

Indice : `factor()` crée un facteur. Utilisez `ordered = TRUE` et le paramètre `levels` pour spécifier l'ordre.

Solution

Module 3 : Fonctions essentielles

Exercice 3.1 : Fonctions mathématiques [Difficulté : Facile]

Énoncé : Soit `x <- c(16, 25, 49, 81, 144)`.

1. Calculez la racine carrée de chaque élément
2. Calculez le logarithme naturel (ln) de chaque élément
3. Calculez le logarithme en base 10
4. Calculez la valeur absolue de `c(-5, 3, -2, 7, -1)`
5. Calculez la somme cumulative de `1:10` avec `cumsum()`

Indice : `sqrt()` pour la racine, `log()` pour ln, `log10()` pour log base 10, `abs()` pour la valeur absolue.

Solution

Exercice 3.2 : Fonctions statistiques [Difficulté : Facile]

Énoncé : Soit `donnees <- c(23, 45, 12, 67, 34, 56, 89, 21, 43, 78)`.

1. Calculez la moyenne, la médiane, l'écart-type et la variance
2. Trouvez le minimum et le maximum
3. Calculez l'étendue (max - min)
4. Trouvez les quantiles (0%, 25%, 50%, 75%, 100%)
5. Calculez la somme et le produit de tous les éléments

Indice : `mean()`, `median()`, `sd()`, `var()`, `min()`, `max()`, `range()`, `quantile()`, `sum()`, `prod()`.

Solution

Exercice 3.3 : Fonctions de chaînes de caractères [Difficulté : Intermédiaire]

Énoncé : Soit `phrase <- "La programmation en R est puissante et flexible"`.

1. Comptez le nombre de caractères
2. Convertissez en majuscules puis en minuscules
3. Extrayez les caractères de la position 4 à 17 avec `substr()`
4. Remplacez "puissante" par "élégante" avec `sub()`
5. Séparez la phrase en mots avec `strsplit()`
6. Collez les mots "R", "est", "génial" avec `paste()`

Indice : `nchar()`, `toupper()`, `tolower()`, `substr()`, `sub()`, `strsplit()`, `paste()`.

Solution

Exercice 3.4 : Opérations logiques et conditions [Difficulté : Intermédiaire]

Énoncé : Soit `ages <- c(15, 22, 17, 30, 12, 45, 18, 16, 25, 19)`.

1. Identifiez les personnes majeures (>= 18)
2. Comptez le nombre de majeurs et de mineurs
3. Identifiez les personnes entre 18 et 30 ans (inclus)
4. Utilisez `which()` pour trouver les positions des majeurs
5. Utilisez `ifelse()` pour créer un vecteur "majeur"/"mineur"
6. Utilisez `any()` et `all()` pour vérifier si au moins un / tous sont majeurs

Indice : `&` est le ET logique, `|` est le OU. `which()` retourne les positions. `ifelse(test, oui, non)` est vectorisé.

Solution

Exercice 3.5 : Valeurs manquantes (NA) [Difficulté : Intermédiaire]

Énoncé : Soit `mesures <- c(23.5, NA, 19.8, 22.1, NA, 25.3, 20.7, NA, 24.6, 21.9)`.

1. Identifiez les positions des valeurs manquantes avec `is.na()`
2. Comptez le nombre de NA

3. Calculez la moyenne en ignorant les NA
4. Remplacez les NA par la moyenne des valeurs non-manquantes
5. Créez un vecteur sans les NA avec `na.omit()`

Indice : La plupart des fonctions statistiques acceptent `na.rm = TRUE` pour ignorer les NA. `na.omit()` supprime les NA.

Solution

Exercice 3.6 : Écrire ses propres fonctions [Difficulté : Intermédiaire]

Énoncé :

1. Écrivez une fonction `celsius_to_fahrenheit(c)` qui convertit Celsius en Fahrenheit
2. Écrivez une fonction `imc(poids, taille)` qui calcule l'indice de masse corporelle (poids en kg, taille en m)
3. Écrivez une fonction `description_stats(x)` qui retourne une liste contenant : moyenne, médiane, écart-type, min, max d'un vecteur numérique
4. Testez chaque fonction

Indice : La syntaxe est `ma_fonction <- function(arg1, arg2) { ... }`. La dernière expression évaluée est la valeur de retour, ou utilisez `return()`.

Solution

Exercice 3.7 : Appliquer des fonctions avec apply/sapply/lapply [Difficulté : Avancé]

Énoncé :

1. Créez une matrice 4x5 de nombres aléatoires entre 1 et 100 (utilisez `set.seed(42)` pour la reproductibilité)
2. Utilisez `apply()` pour calculer la moyenne de chaque ligne
3. Utilisez `apply()` pour calculer l'écart-type de chaque colonne

4. Soit `nombres <- list(a = 1:10, b = 20:30, c = 50:100)`. Utilisez `sapply()` pour calculer la moyenne de chaque élément
5. Utilisez `lapply()` pour obtenir le résumé (`summary`) de chaque élément
6. Utilisez `sapply()` avec une fonction anonyme pour calculer le coefficient de variation (sd/mean x 100) de chaque élément

Indice : `apply(X, MARGIN, FUN)` où MARGIN=1 pour les lignes, MARGIN=2 pour les colonnes. `sapply()` simplifie le résultat, `lapply()` retourne une liste.

Solution